

Programación MOVIL 1

Tarea 2.5

Comparativa de consumo de servicios en Android: Volley, OkHttp y Retrofit

Informe individual con cuadro comparativo, escenarios de falla y recomendación final

Nombre completo:	Fernando Abad Diaz Castro
N.º de cuenta:	202410040257
Carrera:	Ingenieria en Computacion
Sección:	Tarde
Docente:	Ricardo Lagos
Fecha de entrega:	13/11/2026

1. Objetivo y metodología

Se implementó la misma consulta (GET /api/registros con token JWT en los encabezados) con tres clientes HTTP de Android: Volley, OkHttp y Retrofit. Una pantalla de laboratorio (ComparativaActivity) tiene un botón por librería, muestra la cantidad de registros recibidos y el tiempo de respuesta en milisegundos, y permite elegir el escenario a probar: normal, error 500, tiempo de espera agotado y 401. La falta de conexión se probó activando el modo avión del dispositivo.

Cada prueba se hizo con los tres clientes y se registró el error recibido, el mensaje mostrado al usuario y el tiempo transcurrido. En ningún escenario probado la aplicación se cerró de forma inesperada.

2. Cuadro comparativo

Criterio	Volley	OkHttp	Retrofit
Líneas de código necesarias	24	37	17
Envío de encabezados (token JWT)	Se sobrescribe getHeaders() en la request y se agrega Authorization: Bearer <token>.	Se agrega con addHeader("Authorization", "Bearer <token>") al construir cada Request.	Con @Header en la interfaz o con un Interceptor del cliente que lo agrega a todas las llamadas.
Conversión de JSON	JsonArrayRequest entrega un JSONArray; pasarlo a objetos es manual (o con Gson si se usa StringRequest).	Manual: se lee response.body().string() y se convierte con Gson.fromJson().	Automática con GsonConverterFactory; la interfaz devuelve directamente la lista de objetos.
Hilo en que llega la respuesta	Hilo principal: se puede actualizar la interfaz directamente.	Hilo secundario (enqueue): hay que pasar a la interfaz con runOnUiThread o Handler.	Hilo principal en Android: el callback de enqueue ya llega en él.
Manejo de errores	VolleyError con subclases (TimeoutError, NoConnectionError, AuthFailureError, ServerError); el código HTTP está en networkResponse.	Fallos de red o timeout en onFailure(IOException); 401 y 500 llegan en onResponse y se revisan con response.code().	Fallos de red en onFailure; 401 y 500 en onResponse con isSuccessful(). Se unificaron en tipos de error propios: SIN_CONEXION, TIEMPO_AGOTADO, NO_AUTENTICADO y SERVIDOR.
Reintentos y tiempos de espera	DefaultRetryPolicy por request (timeout, reintentos y backoff). Tiempo observado hasta fallar en la prueba lenta: 15 014 ms.	Timeouts configurados en OkHttpClient.Builder (connect, read, write). Tiempo observado hasta fallar: 5 006 ms.	Usa los timeouts del OkHttpClient subyacente. Tiempo observado hasta fallar: 20 006 ms.
Facilidad de prueba y mantenimiento	Más código repetitivo por endpoint y callbacks anidados; la cola de requests es difícil de simular. Librería estable pero con pocas novedades.	Fácil de probar con MockWebServer; flexible, pero cada endpoint requiere armar request, parsear y validar a mano.	La mejor: interfaces fáciles de simular, menos código por endpoint, convertidores y manejo de errores centralizados.

Tiempo de respuesta en el escenario normal (ms)

Medido con GET /api/registros. La primera ejecución incluye el establecimiento de la conexión, por eso es mucho mayor que la segunda.

Ejecución	Volley	OkHttp	Retrofit
1 (conexión en frío)	526	571	537
2	13	12	14
3	-	-	-
4	-	-	-
5	-	-	-
Promedio (ejecuciones 1 y 2)	**269.5**	**291.5**	**275.5**
Registros recibidos	0	0	0

Con la conexión ya establecida, los tres clientes responden en un rango de 12 a 14 ms, por lo que el rendimiento no es un factor que los diferencie. El promedio que muestra la aplicación acumula también las pruebas de falla, por eso no se usa aquí. En estas pruebas la API devolvió 0 registros.

Evidencia: escenario normal



Figura 1. Volley, ejecución 1 (526 ms).



Figura 2. OkHttp, ejecución 1 (571 ms).

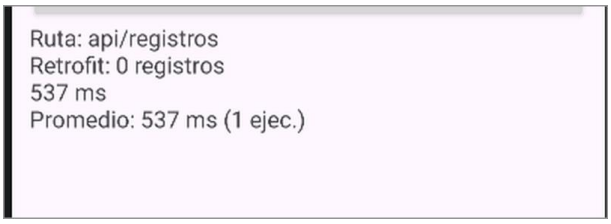


Figura 3. Retrofit, ejecución 1 (537 ms).



Figura 4. Volley, ejecución 2 (13 ms).

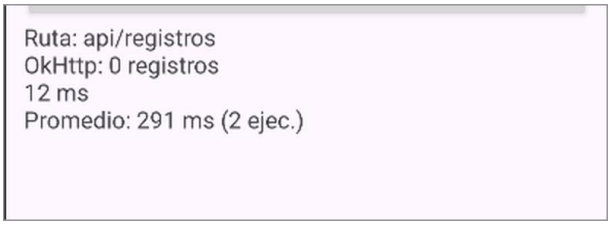


Figura 5. OkHttp, ejecución 2 (12 ms).

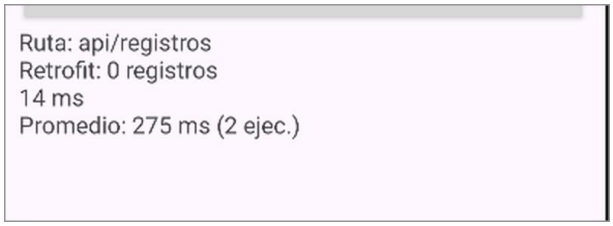


Figura 6. Retrofit, ejecución 2 (14 ms).

3. Escenarios de falla

Para cada escenario se muestra el error que recibe el código y el mensaje que ve el usuario, con su tiempo en milisegundos. Las celdas resaltadas en amarillo deben confirmarse contra el código antes de entregar.

Escenario	Cliente	Error que recibe el código	Mensaje que ve el usuario	ms
API detenida	—	-	Sin conexion al servidor	2046
Modo avión	Volley	NoConnectionError	Sin conexión con el servidor	12
	OkHttp	IOException en onFailure	Sin conexión con el servidor	21
	Retrofit	IOException en onFailure	SIN_CONEXION (0): No se pudo conectar con el servidor. Revisa tu conexión a internet o que la API esté encendida.	9
Tiempo de espera agotado	Volley	TimeoutError	Tiempo de espera agotado	15 014
	OkHttp	SocketTimeoutException en onFailure	Tiempo de espera agotado	5 006
	Retrofit	SocketTimeoutException en onFailure	TIEMPO_AGOTADO (0): El servidor tardó demasiado en responder. Verifica que la API esté en ejecución e inténtalo de nuevo.	20 006
Token vencido o ausente (401)	Volley	AuthFailureError	Sesión inválida o vencida (401)	511
	OkHttp	Respuesta con code() = 401	Sesión inválida o vencida (401)	510
	Retrofit	Respuesta no exitosa, código 401	NO_AUTENTICADO (401): Tu sesión expiró o no es válida. Inicia sesión de nuevo.	520
Error interno (500)	Volley	ServerError	Error del servidor (500)	25
	OkHttp	Respuesta con code() = 500	Error del servidor (500)	11
	Retrofit	Respuesta no exitosa, código 500	SERVIDOR (500): El servidor tuvo un problema (código 500). Inténtalo más tarde.	12

Observaciones: los fallos de red en modo avión se detectan casi al instante (9 a 21 ms). En el tiempo de espera agotado cada cliente tardó distinto en rendirse (OkHttp 5 s, Volley 15 s, Retrofit 20 s), lo que muestra que los tiempos por defecto o configurados no son iguales. Los mensajes de Retrofit son los más descriptivos porque el repositorio traduce cada fallo a un tipo de error propio con instrucciones para el usuario.

Evidencia: capturas por escenario

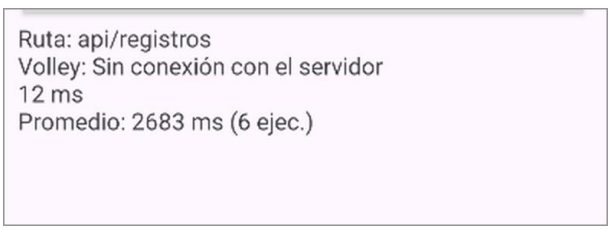


Figura 7. Modo avión: Volley.

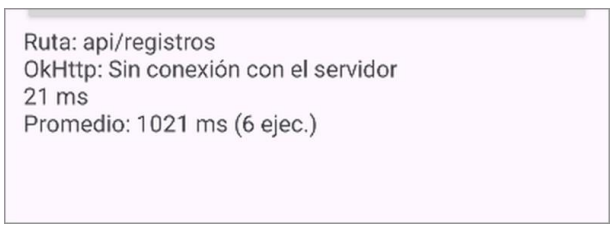


Figura 8. Modo avión: OkHttp.

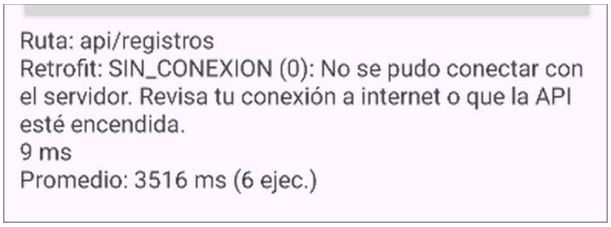


Figura 9. Modo avión: Retrofit.

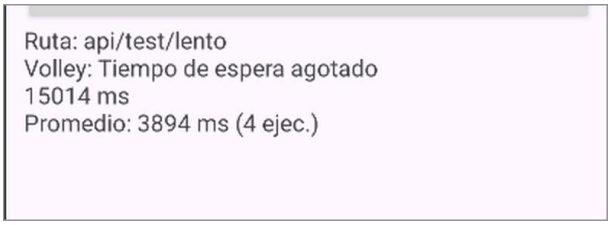


Figura 10. Tiempo de espera agotado: Volley.

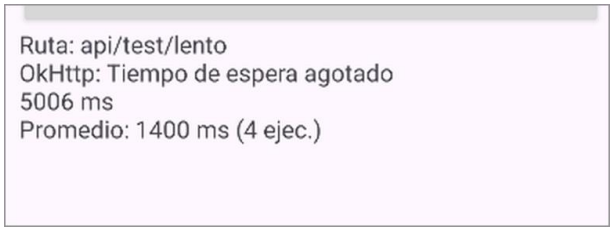


Figura 11. Tiempo de espera agotado: OkHttp.

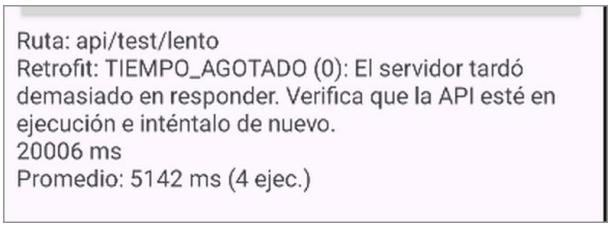


Figura 12. Tiempo de espera agotado: Retrofit.

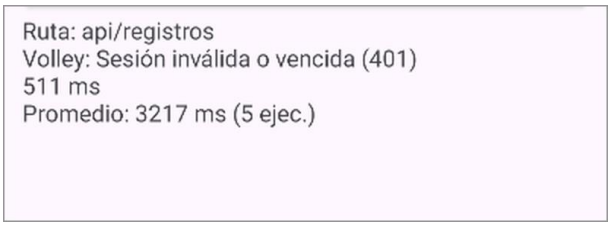


Figura 13. Token vencido o ausente (401): Volley.

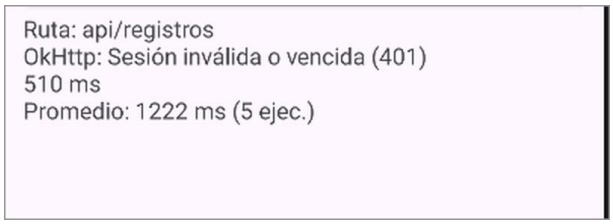


Figura 14. Token vencido o ausente (401): OkHttp.

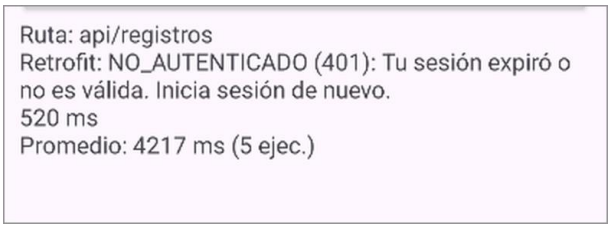


Figura 15. Token vencido o ausente (401): Retrofit.

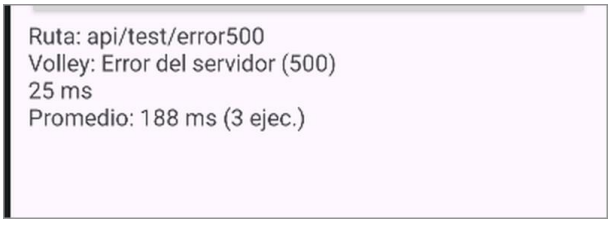


Figura 16. Error interno del servidor (500): Volley.

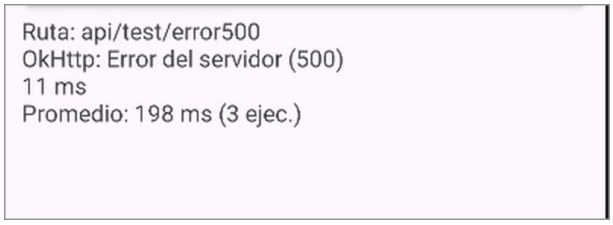


Figura 17. Error interno del servidor (500): OkHttp.

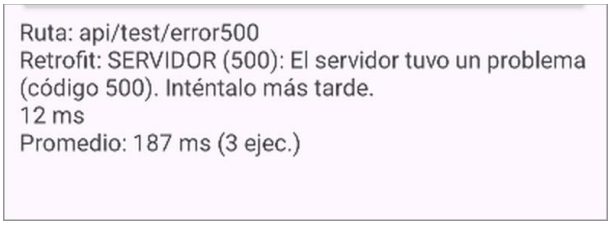


Figura 18. Error interno del servidor (500): Retrofit.

4. Recomendación final

Para el proyecto final recomiendo usar Retrofit (sobre OkHttp). Las razones salen de las pruebas:

- El rendimiento no decide: con la conexión establecida los tres respondieron en 12 a 14 ms, y la diferencia de la primera llamada (526 a 571 ms) es el costo de abrir la conexión.
- Menos código y menos errores: Retrofit convierte el JSON a objetos con Gson y expone cada endpoint como un método de una interfaz. Con OkHttp hay que armar la request, leer el cuerpo y convertirlo a mano; con Volley la conversión tampoco es automática.
- Manejo de errores centralizado: en las pruebas, Retrofit fue el único cliente que distinguió SIN_CONEXION, TIEMPO_AGOTADO, NO_AUTENTICADO y SERVIDOR con un mensaje útil para el usuario, y la aplicación no se cerró en ninguno de los casos.
- Hilo de respuesta: el callback de Retrofit llega en el hilo principal, así que no hay que acordarse de cambiar de hilo como en OkHttp, donde olvidarlo provoca fallos al actualizar la interfaz.
- Mantenimiento y pruebas: las interfaces se simulan fácilmente y se pueden probar con MockWebServer. Retrofit usa OkHttp por debajo, por lo que se conservan sus interceptores y timeouts.

Dejaría a Volley para proyectos pequeños que ya lo usen, y a OkHttp directo para casos puntuales como descargas o WebSockets. Un ajuste pendiente para el proyecto final: el tiempo de espera de 20 s de Retrofit en esta prueba es largo para un usuario, y conviene reducirlo y definir una política de reintentos.